1. SYNOPSIS

Project Title: Fraud URL Detection

Group No:12

Internal Guide: Prof. Dumbre.S. A

Technical Keywords: Cyber security, Phishing, Hacking, Fraud, etc.

1.1 Relevant Objectives:

In recent days, the demand for cyber security and protection against various types of cyber-attacks has been ever increasing. It is essential to prevent and protect our data from such hackers. Therefore, we have created an project which will help to detect such kind of attacks. The project will help to detect whether our application has been attacked or not so that the user/customer will know that their data is been stolen & they can take action according

1.2 Motivation:

Cyber-attacks refer to those attacks launched on unsuspecting online users either using a computer as the object of the crime (hacking, phishing, spamming etc.), or as a tool to advance other criminal activities (cyber stalking, identity theft, child pornography etc.). Cyber-attacks are increasing exponentially hence making cyber security to be a challenge in this digital age. Cyber-attacks when successfully launched can result in monumental losses to businesses and individual hence quick incident responses are required to salvage the situation in case of an occurrence of cyber attacks

1.3 Hypothesis:

Hypothesis 1: International crime drop (Farrell and Birks 2018) we reiterate this hypothesis- which we derived from the work of varying scholars quoted in our original manuscript.

Hypothesis 2: The criticism relating to cybercrime was that some crimes, such as fraud, may have adopted online modus operandi in recent years.

Hypothesis 3: our critics show the online banking fraud in the UK increased from declined from which they infer substitution by offenders who preferred online crime whatever reason.

Hypothesis 4: Our principles supposition was that the spread of the Internet occurred too late to account for the major declines in crime experienced across high income countries.

1.4 Plan of Project Execution:

Task	Time Duration (In Days)						Total Days
	15	30	45	60	75	90	
Requirement Gathering & Analysis							15
Designing							15
Coding							35
Testing							15
Deployment & Implementation							10
Documents							80
Total							90

Technical Keyword

Cyber Security:

Cybersecurity is the art of protecting networks, devices, and data from unauthorized access or criminal use and the practice of ensuring confidentiality, integrity, and availability of information. It seems that everything relies on computers and the internet now—communication (e.g., email, smartphones, tablets), entertainment (e.g., interactive video games, social media, apps), transportation (e.g., navigation systems), shopping (e.g., online shopping, credit cards), medicine (e.g., medical equipment, medical records), and the list goes on. How much of your daily life relies on technology? How much of your personal information is stored either on your own computer, smartphone, tablet or on someone else's system.

Hacking:

is the activity of identifying weaknesses in a computer system or a network to exploit the security to gain access to personal data or business data? An example of computer hacking can be: using a password cracking algorithm to gain access to a computer system.

Fraud:

Fraud is an intentionally deceptive action designed to provide the perpetrator with an unlawful gain or to deny a right to a victim. Types of fraud include tax fraud, credit card fraud, wire fraud, securities fraud, and bankruptcy fraud. Fraudulent activity can be carried out by one individual, multiple individuals or a business firm as a whole.

2. INTRODUCTION

Online services have become an irreplaceable part of today's businesses, schools, banking, or personal lives. With their increasing popularity, the number of malicious websites is growing. A malicious website contains some unsolicited content with a purpose to gather sensitive data or install malware onto a user's machine. Usually, some interaction from the user part is needed, but in the case of a drive by download, malware is installed automatically without asking for permission

Prevention from such attacks is complicated, as being cautious is sometimes not enough. Attackers can be able to exploit vulnerabilities in web applications to insert malicious code without knowledge of the owner. According to the 2019 Web root Threat Report 40% of malicious URLs were found on good domains. On the first look, legitimate websites can become a potential threat for users. Browser and anti-viruses re expected to prevent a user from accessing such websites.

The URL is usually the first and cheapest information we have about a website. Therefore, it is a natural choice to develop techniques that would recognize a malicious URL from benign. Moreover, accessing and downloading the content of the website can be time consuming and brings risks associated with downloading potentially harmful content.

Malicious URLs URL is the abbreviation of Uniform Resource Locator, the global address of documents and other resources on the World Wide Web. In other words, it is a specific character string which constitutes a reference to an existing Internet resource. A URL usually consist of three to five components: scheme, identifies the protocol that should be used host, specifies the IP address or registered domain name path, identifies the location of a specific resource in the host, can be followed by port number query string, a string of parameters and values, part of the URL coming after'?' fragment, identification of a secondary resource, usually a subsection within the page

URLs features Using the whole URL string as only information for detecting URLs may not be enough. Therefore, necessarily first step is selecting and extracting useful features that sufficiently describe the URL. Working with features and not using URL as a whole string would allow creating more effective and successful detection algorithms. For each type of malicious URLs, a different set of features can bring better results. Papers addressing malicious

URL detection have proposed several types of features that can be used. We can categorize these features into Blacklist, Lexical, Host-based, Content-based features, and Rank-based features

Blacklist features Blacklist is essentially a listing of URLs that were previously identified as malicious. It is continuously updating over time using a combination of automatic mechanisms and humans. Using blacklist for identification of malicious URLs is considered as very fast and easy technique to implement with a very low falsepositive1 rate. Unfortunately, this technique fails to detect newly generated URLs, and therefore suffering from a high false-negative2 rate.

However, instead of using blacklist as a solo identification tool, it can be a handy feature for more advanced techniques. Detection tools can use their own created blacklists, or because of high data storage capacity needed, one of many public blacklisting APIs such as from Google Safe Browsing or Phish Tank

1. False positive – benign URL is misclassified as malicious
2. False negative – malicious URL is undetected.

Lexical features Lexical features idea is based on the assumption that malicious websites look different and thus have distinguishable patterns in their URL text. In many cases is a user able to recognize suspicious URLs just from the look. This applies mostly for phishing URLs, where the URL tends to look similar as of the original page it claims to be.

For example, taking the URLs: https://www.paypal.com/ and https://paypal.co.uk.b8wt.icu/n/ The second URL has from the first sight too many domain names and looks suspicious. Under lexical features, we refer to all textual properties of URL itself, not including the content of the page it points to. Lexical features are attractive to use because they are fast to process, need low amounts of data storage, and can be obtained without having to call other services. Common detection techniques usually treat lexical features of host name and path separately, because of their different impact on classification. According to Sahoo, Liu and Hoi we can divide lexical features to Traditional and Advanced one.

Traditional lexical features Traditional lexical features include all commonly used lexical features. Most of them are a combination of features motivated by McGrath and Gupta and Kolari et al. This includes the length of the hostname, the length of the entire URL, as well as the frequencies of the characters in the URL. Further, the presence or order of tokens in the hostname (delimited by '.') and in the path (delimited by '.', '/', '?', '=', '-', ' ') are used. Advanced lexical features Advanced lexical features need more computation and are more complex than traditional ones. However, they are more successful in the detection of various obfuscation techniques. Typical examples of obfuscation techniques are

(a) host replaced with an IP address,
(b) targeted domain in the path
(c) large hostnames, additional domains
(d) domain unknown or misspelled.

Lexical features Lexical features idea is based on the assumption that malicious websites look different and thus have distinguishable patterns in their URL text. In many cases is a user able to recognize suspicious URLs just from the look. This applies mostly for phishing URLs, where the URL tends to look similar as of the original page it claims to be.

For example, taking the URLs: https://www.paypal.com/ and https://paypal.co.uk.b8wt.icu/n/ The second URL has from the first sight too many domain names and looks suspicious. Under lexical features, we refer to all textual properties of URL itself, not including the content of the page it points to. Lexical features are attractive to use because they are fast to process, need low amounts of data storage, and can be obtained without having to call other services. Common detection techniques usually treat lexical features of host name and path separately, because of their different impact on classification. According to Sahoo, Liu and Hoi we can divide lexical features to Traditional and Advanced one.

3.PROBLEM DEFINITION & SCOPE

PROBLEM DEFINITION:

We provide a dataset that can be useful for evaluating the performance of a classifier for aforementioned malicious URLs. Obfuscation techniques are commonly used by spammers and scammers in order to obscure (hide and confuse) any malicious URL. However, the malicious owner can also use these obfuscation techniques to evade detection and tempt users to hit. Therefore, so et al. proposes to dissect URLs to several portions and use logistic regression for detection. Apart from the work of Choi et al. where authors evaluate the performance of a classifier for discriminating three types of malicious URLs (Spam, Phishing, Malware), most of the related works focus on either malicious URLs in general or any specific type of URLs. Regarding feature selections, most of the works depend on various kind of features: lexical, content, obfuscation, DNS etc. In this work, we consider four different types of malicious URLs (Defacement, Spam, Phishing, Malware) for experiments. We are applying only static lexical features for classifying URLs (to achieve high performance), our malicious URL detector produces a promising as well as competitive performance to some existing works.

Social engineering attack is a common security threat used to reveal private and confidential information by simply tricking the users without being detected. The main purpose of this attack is to gain sensitive information such as username, password and account numbers. According to, phishing or web spoofing technique is one example of social engineering attack. Phishing attack may appear in many types of communication forms such as messaging, SMS, VOIP and fraudster emails. Users commonly have many user accounts on various websites including social network, email and also accounts for banking. Therefore, the innocent web users are the most vulnerable targets towards this attack since the fact that most people are unaware of their valuable information, which helps to make this attack successful. Typically, phishing attack exploits the social engineering to lure the victim through sending a spoofed link by redirecting the victim to a fake web page. The spoofed link is placed on the popular web pages or sent via email to the victim. The fake webpage is created similar to the legitimate webpage. Thus, rather than directing the victim request to the real web server, it will be directed to the attacker server. The current solutions of antivirus, firewall and designated software do not fully prevent the web spoofing attack. The implementation of Secure Socket Layer (SSL) and digital certificate (CA) also does not protect the web user against such attack. In web

spoofing attack, the attacker diverts the request to fake web server. In fact, a certain type of SSL and CA can be forged while everything appears to be legitimate. According to, secure browsing connection does virtually nothing to protect the users especially from the attackers that have knowledge on how the “secure” connections actually work. This paper develops an anti-web spoofing solution based on inspecting the URLs of fake web pages. This solution developed series of steps to check characteristics of websites Uniform Resources Locators (URLs).

SCOPE:

The success of these techniques highly depends on the quality and combination of relevant characteristic features of web pages which may include network traffic information, content characteristics, lexical features of URLs, and even domain name system (DNS) information. However, extracting these attributes may be costly and sometimes require downloading complete web pages or looking up various DNS servers and ISPs to get enrichment data like geo-location, registration records, and network information, which face the problem of network latency, making them impractical for real-time systems. Due to the difficulties of using non-lexical features to detect malicious URLs in real-time despite their high accuracy values, previous works have explored the use of URL lexical features only and it has been proven that URL features alone can produce an accurate means of detecting malicious webpage in real time systems.

However, extracting these attributes may be costly and sometimes require downloading complete web pages or looking up various DNS servers and ISPs to get enrichment data like geo-location, registration records, and network information, which face the problem of network latency, making them impractical for real-time systems. Due to the difficulties of using non-lexical features to detect malicious URLs in real-time despite their high accuracy values, previous works have explored the use of URL lexical features only and it has been proven that URL features alone can produce an accurate means of detecting malicious webpage in real time systems.

4.OBJECTIVES AND GOALS

OBJECTIVES:

The basic goal for most fraud examinations is to determine whether fraud occurred, and if so, who perpetrated it. A particular engagement may, however, have additional goals, such as to establish and secure evidence to be used in a criminal or other disciplinary action or to provide proof to recover losses from an insurer (2017 Fraud Examiners Manual). The objective in an audit of financial statements is to determine whether they are free of material misstatement, regardless of whether that misstatement is intentional or not; in other words, a fraud examiner's priority is proving the nature and extent of a particular fraud, but an auditor's focus is detecting material misstatements. Implicit in this difference are several other naturally resulting differences related to scope, methodology and professional standards, and the relationship to stakeholders.

The main reason is the lack of awareness of users. But security defenders must take precautions to prevent users from confronting these harmful sites. Preventing these huge costs can start with making people conscious in addition to building strong security mechanisms which are able to detect and prevent phishing domains from reaching the user.

GOALS:

1) Python can be used to develop prototypes, and quickly because it is so easy to work with and read.

2) Most automation, data mining, and big data platforms rely on Python.

3) Python allows for a more productive coding environment than massive languages like C# and Java. Experienced coders tend to stay more organized and productive when working with Python

4) Python is easy to read, even if you're not a skilled programmer. Anyone can begin working with the language, all it takes is a bit of patience and a lot of practice. Plus, this makes it an ideal candidate for use among multi-programmer and large development teams.

5) Python powers Django, a complete and open-source web application framework. Frameworks - like Ruby on Rails - can be used to simplify the development process.

6) It has a massive support base thanks to the fact that it is open source and community developed. Millions of like-minded developers work with the language on a daily basis.

5.PROJECT PLAN

5.1 METHODOLOGY:

Using lexical features with a proposed classification engine gives high accuracy in classifying the URLs. Domains used for phishing purpose have shown to have different lengths and diverse location. McGrath et al. analyse the differences between benign and phishing URLs using features such as URL and domain length. Based on the behaviour of domains in phishing websites, Ma et al. manage to identifying suspicious URLs by using lexical and blacklisted host-based features. They believe that there are certain red flag keywords tend to appear in malicious URLs such as appearing ebayisapi for spoofing ebay web pages. However, in the following works, Ma et al.use supervised learning across both lexical and host-based features. In this case, authors argue that their approach is complementary to both blacklisting and system-based evaluation such as site content and behaviour analysis where one cannot predict seeing only the status of previously unseen URLs. Moreover, this requires visiting potentially dangerous sites.

They figure out that using appropriate classifiers, it is possible to identify most predictive features for automatically classifying malicious or benign URLs. authors use lexical, host and page-content based features for identifying malicious URLs collected from spam messages in twitter and emails. Choi et al. in, present a machine learning method to detect malicious URLs and identify attack types such as spamming, phishing, and malware.

Their studies include multiple types of malicious URLs using features from six different areas namely lexicon, link popularity, web page content, DNS, DNS fluxing and network traffic. However, their result shows that using lexical features yield lower accuracy for spam and malware URL dataset. Authors in use descriptive features of URL to complement lexical features. They combine the lexical information and static characteristics of URL string to classify malicious URLs. Without host and content-based analysis, in this experiment, they were able to deal with two million URLs in five minutes and their proposed method misses around 9 % of malicious instances.

The effectiveness of machine learning based phishing detection with known protected websites has been studied by Chu et al. Based on only lexical and domain features authors propose several highly effective features with detection rate over 91 %. authors study hidden

fraudulent URLs which are embedded to the trusted URLs and also defacement URLs which are legitimate pages belonging to trusted but compromised web sites.

5.2 Feasibility Study:

After the problem is clearly understood and solutions proposed, the next step is to conduct the feasibility study. Feasibility study is defined as evaluation or analysis of the potential impact of a proposed project or program. The objective is to determine whether the proposed system is feasible. There are three aspects of feasibility study to which the proposed system is subjected as discussed below.

5.3 Technical Feasibility

Technical feasibility assesses whether the current technical resources are sufficient for the new system. If they are not available, can they be upgraded to provide the level of technology necessary for the new system? It checks whether the proposed system can be implemented in the present system without supporting the existing hardware. Since the application need only android smartphone as an external hardware the software is technically feasible.

5.4 Economic Feasibility:

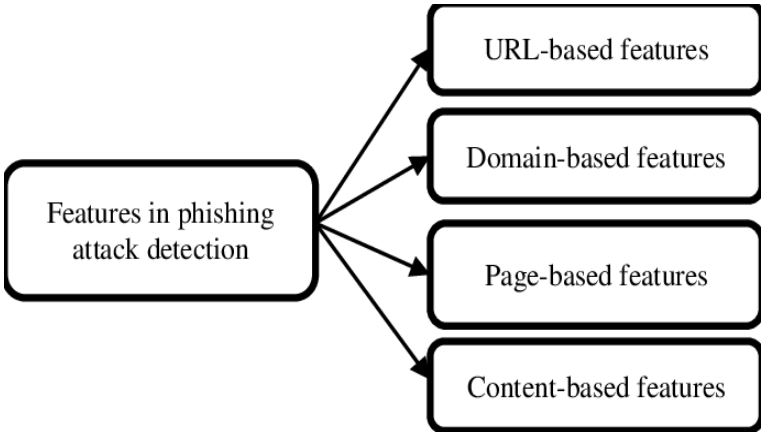
Economic feasibility determines whether the time and money are available to develop the system. It also includes the purchase of new equipment, hardware, and software. A software product must be cost effective in the development, on maintenance and in the use. Since the hardware and resources are already available with the organization and the organization can afford to allocate the required resources.

5.5 Operational Feasibility:

Operational feasibility determines if the human resources are available to operate the system once it has been installed. The resources that are required to implement or install are already available with the organization. The persons of the organization need no exposure to computer but have to be trained to use this particular software. A few of them will be trained. Further, training is very less. The management will also be convinced that the project is optimally feasible.

What are malicious attacks?

We all are familiar with the fact that our pc, laptops, Tabs, and mobile phones get attacked by viruses or malware, and to prevent and protect our devices from such attacks, we install various protective solutions that are either paid or licensed, or free (but for a limited time, trial version). This is about the safe practice we implement in layman's terms, but most people aren't aware of malicious attacks? And what are its types? Let’s discuss malicious attacks to gain some knowledge about these guys who are unauthorized guests!



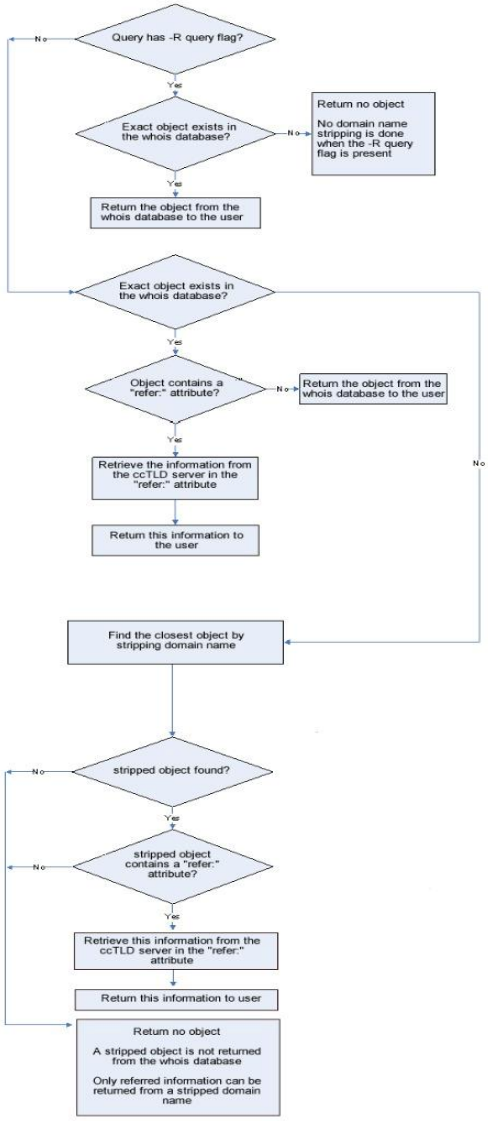
5.6 MODULES:

SKLEARN

Scikit-learn provides a range of supervised and unsupervised learning algorithms via a consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use. The library is built upon the SciPy (Scientific Python) that must be installed before you can use scikit-learn. This stack that includes: • NumPy: Base n-dimensional array package • SciPy: Fundamental library for scientific computing • Matplotlib: Comprehensive 2D/3D plotting • IPython: Enhanced interactive console • Sympy: Symbolic mathematics • Pandas: Data structures and analysis Extensions or modules for SciPy care conventionally named SciKits. As such, the module provides learning algorithms and is named scikit-learn. The vision for the library is a level of robustness and support required for use in production systems. This means a deep focus on concerns such as ease of use, code quality, collaboration, documentation and performance. 10 Although the interface is Python, c-libraries are leverage for performance such as NumPy for arrays and matrix operations, LAPACK, LibSVM and the careful use of python. The library is focused on modelling data. It is not focused on loading, manipulating and summarizing data. For these features, refer to NumPy and Pandas.

5.7 WHOIS

The life of phishing site is very short, therefore; this DNS information may not be available after some time. If the DNS record is not available anywhere then the website is phishing. If the domain name of the suspicious webpage is not match with the WHOIS database record, then webpage considers as phishing.



5.8 PANDAS:

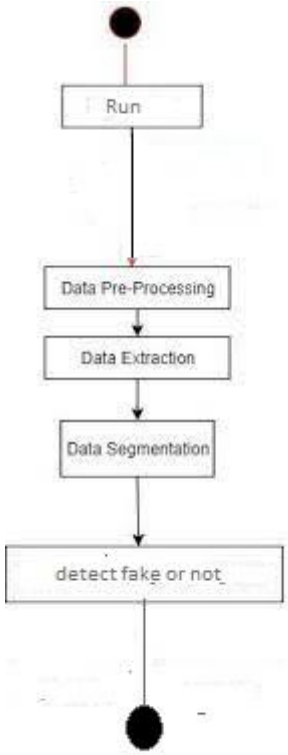
pandas are a Python package that provides fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real world data analysis in Python. Additionally, it has the broader goal of becoming the most powerful and flexible open-source data analysis / manipulation tool available in any language. It is already well on its way towards this goal.

- Easy handling of missing data (represented as NaN, NA, or NaT) in floating point as well as non-floating-point data
- Size mutability: columns can be inserted and deleted from Data Frame and higher dimensional objects
- Automatic and explicit data alignment: objects can be explicitly aligned to a set of labels, or the user can simply ignore the labels and let Series, Data Frame, etc. automatically align the data for you in computations
- Powerful, flexible group by functionality to perform split-apply-combine operations on data sets, for both aggregating and transforming data
- Make it easy to convert ragged, differently-indexed data in other Python and NumPy data structures into Data Frame objects
- Intelligent label-based slicing, fancy indexing, and sub setting of large data sets
- Intuitive merging and joining data sets
- Flexible reshaping and pivoting of data sets
- Hierarchical labelling of axes (possible to have multiple labels per tick)
- Robust IO tools for loading data from flat files (CSV and delimited), Excel files, databases, and saving/loading data from the ultrafast HDF5 format
- Time series-specific functionality

6. SYSTEM DESIGN AND SPECIFICATION

6.1 Activity Diagram:

Activity diagram is basically a flowchart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. The control flow is drawn from one operation to another. This flow can be sequential, branched, or concurrent.



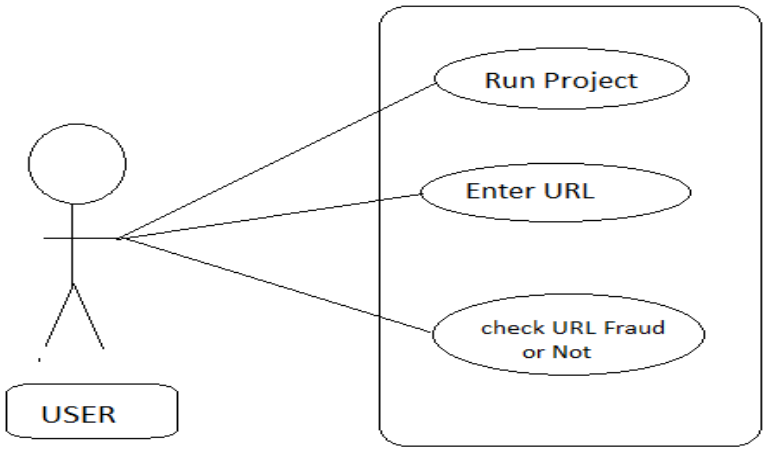
6.2 DFD Diagram

DFD is the abbreviation for **Data Flow Diagram**. The flow of data of a system or a process is represented by DFD. It also gives insight into the inputs and outputs of each entity and the process itself. DFD does not have control flow and no loops or decision rules are present



6.3 Use Case diagram:

A use case diagram is a graphical depiction of a user's possible interactions with system. A use case diagram shows various use cases and different types of users the system has and will often be accompanied by other types of diagrams as well. The use cases are represented by either circles or ellipses



7.PERFORM EVALUATION & TESTING

7.1 WORKING:

Phishing is a form of fraud in which the attacker tries to learn sensitive information such as login credentials or account information by sending as a reputable entity or person in email or other communication channels.

Cybercriminals use phishing URLs to try to obtain sensitive information for malicious use, such as usernames, passwords, or banking details. They send phishing emails to direct their victims to enter sensitive information on a fake website that looks like a legitimate.

Detecting Phishing Domains is a classification problem, so it means we need labeled data which has samples as phish domains and legitimate domains in the training phase. The dataset which will be used in the training phase is a very important point to build successful detection mechanism. We have to use samples whose classes are precisely known. So, it means, the samples which are labeled as phishing must be absolutely detected as phishing. Likewise, the samples which are labeled as legitimate must be absolutely detected as legitimate. Otherwise, the system will not work correctly if we use samples that we are not sure about. Collecting legitimate domains is another problem. For this purpose, site reputation services are commonly used. These services analyse and rank available websites. This ranking may be global or may be country-based. Ranking mechanism depends on a wide variety of features. The websites which have high rank scores are legitimate sites which are used very frequently. One of the well-known reputations ranking service is Alexa. Researchers are using top lists of Alexa for legitimate sites.

7.2 UNIT TESTING:

Fraud detection is a set of processes and analyses that allow businesses to identify and prevent unauthorized financial activity. This can include fraudulent credit card transactions, identify theft, cyber hacking, insurance scams, and more. This chapter presents the unit test cases developed to test one of the modules of the ISOT phishing detection system. The test cases were developed using a test automation framework. An overview of the unit testing and the test automation framework used in the project are provided. Then, several examples of test cases are presented. Unit testing is a method by which individual units of source code are tested separately. The main goal of unit testing is to isolate each part of the program and test that the individual parts, methods, or classes are working correctly. It allows automation of the testing

process and reduces difficulties in discovering bugs in the system or application. This means that a set of inputs given to a function should return the correct values which can be determined by comparing the returned and expected values.

7.3 MODULE UNDER TESTING:

One of the modules of the ISOT phishing detection system, the URL verifier, is The URL verifier involves subsystems that use different techniques to detect phishing URLs. One of the subsystems uses machine learning classification. Another subsystem uses heuristic rules to detect these URLs. One of the tasks in this project is to generate and execute unit test cases for the heuristic rule-based subsystem. This subsystem implements a collection of rules that extract different features from URLs embedded in a message body and determine whether they are legitimate or phishing. Unit testing is a method by which individual units of source code are tested separately. The main goal of unit testing is to isolate each part of the program and test that the individual parts, methods, or classes are working correctly. It allows automation of the testing process and reduces difficulties in discovering bugs in the system or application. This means that a set of inputs given to a function should return the correct values which can be determined by comparing the returned and expected values.

7.4 TESTING FRAMEWORK:

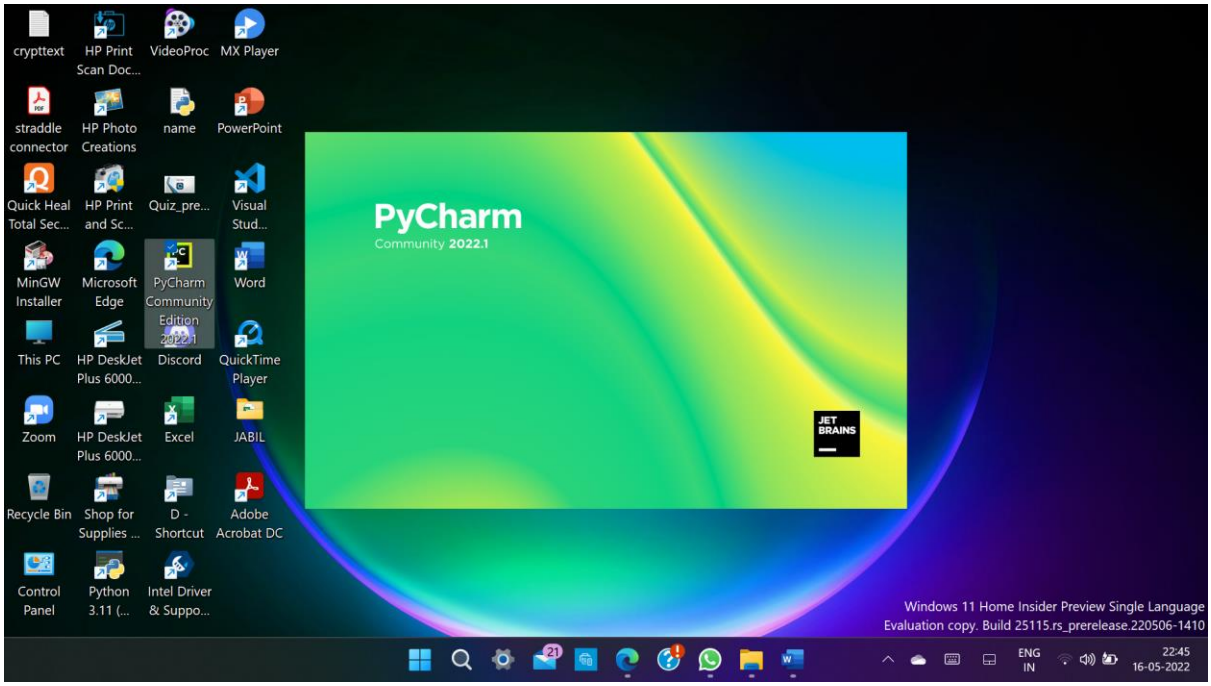
A testing framework is defined as a set of assumptions, concepts, and practices that constitute a work platform or support for testing or automated testing. The testing framework is responsible for defining the format in which to express expectations, execution results and reporting results. There are various testing frameworks and tools for the Java programming language such as JUnit, Selenium, REST-Assured, Robot-framework, Mockito, and TestNG. In this project, the JUnit framework is used. JUnit is a unit testing framework for the Java programming language. It plays an important role in test-driven development and it is part of the family of unit testing frameworks known as Unit that originated with JUnit. JUnit is used to set up the test data for a piece of code before it is implemented. This increases the productivity of the programmer and the stability of the code, and reduces the time spent on debugging. The features of JUnit are as follows.

8. RESULT AND ANALYSIS

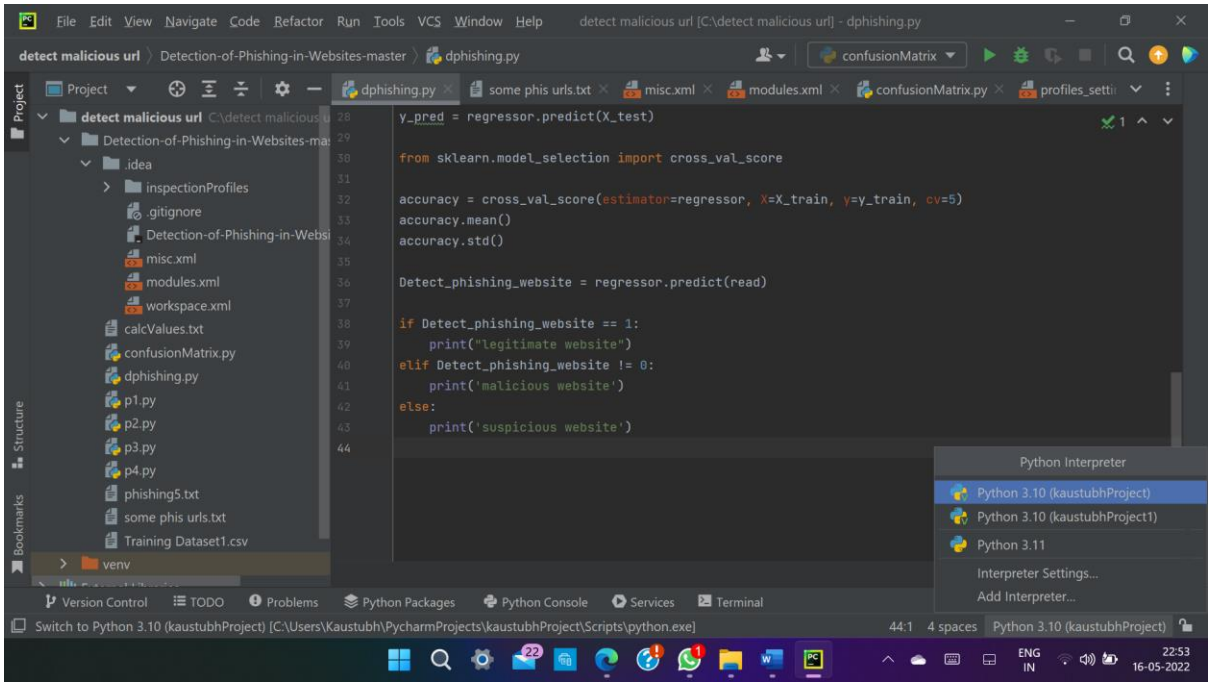
Always perform a SWOT analysis to identify what is fraud detection and the needs of the organization. plan for a dedicated fraud management team. Use the most suitable buy or build option for the system. Clean all data before using it. Make relevant business rules for the layout of the system. Set threshold values of transactions to avoid fraudulent transactions getting through. Use the Predictive Modelling features for planning. Try the use of the SNA when implementing solutions bought. Leverage social media to build integrated case management systems. Build forward-looking analytics solutions**.**

8.1 OUTPUTS:

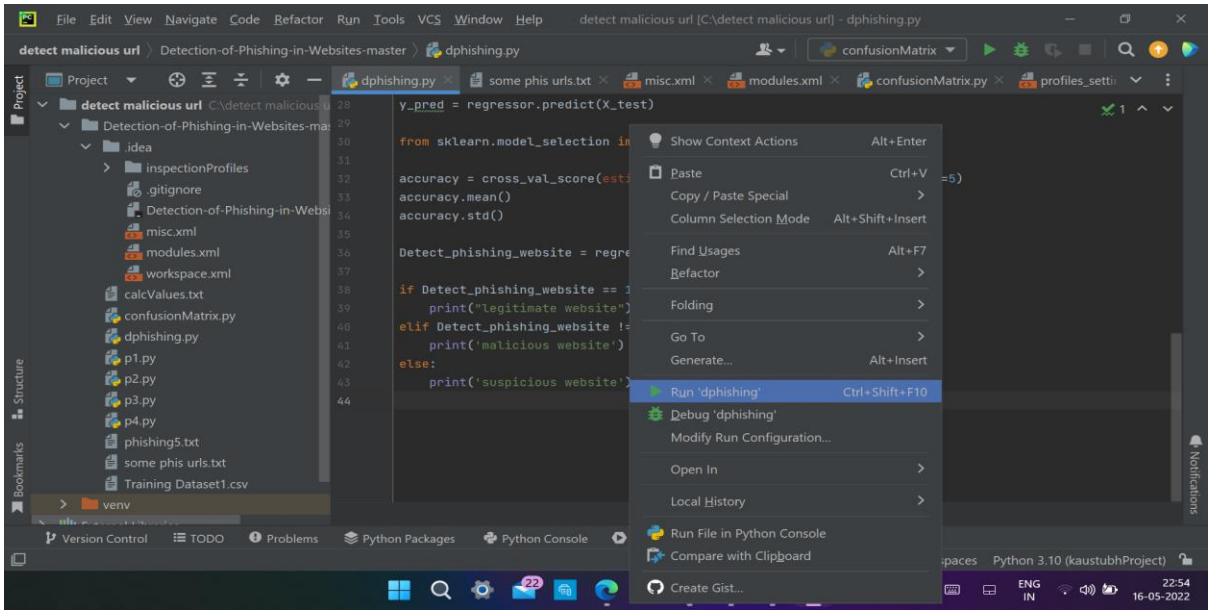
A. Open the PyCharm:



B. Check whether python interpreter is set with PyCharm otherwise code will not run:

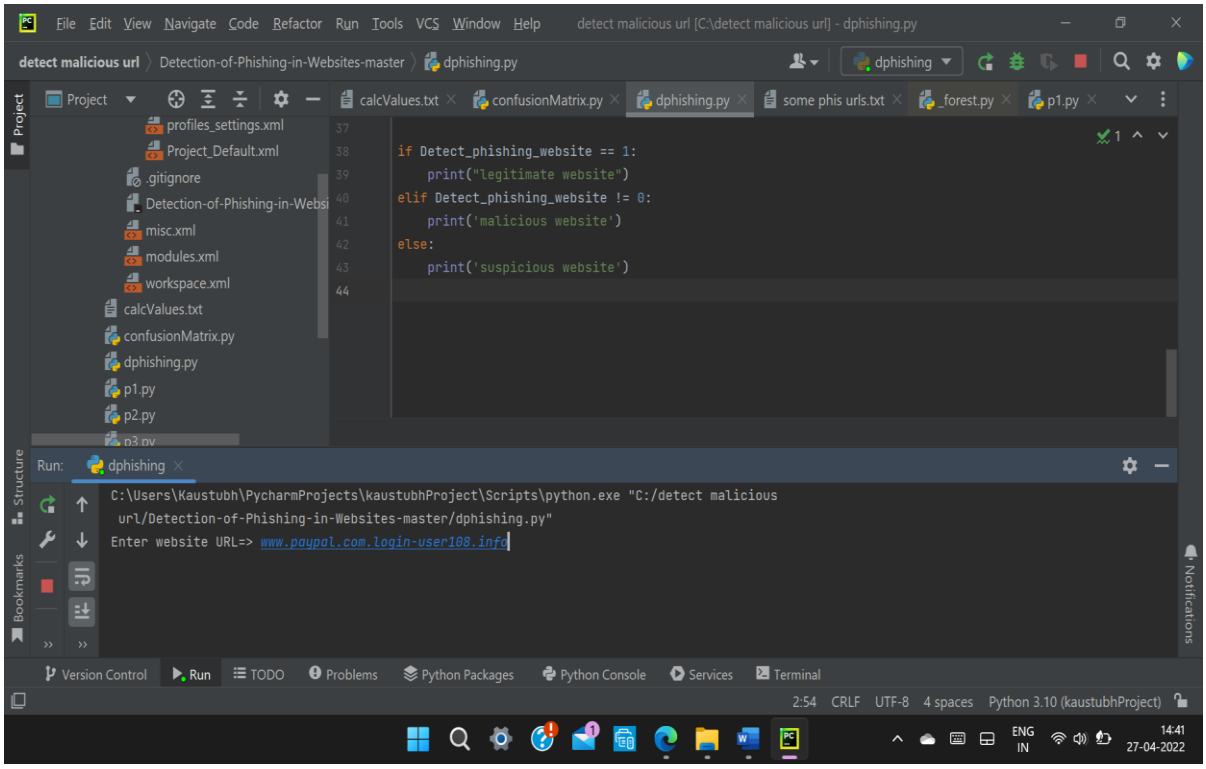


C. After that run the dphishing.

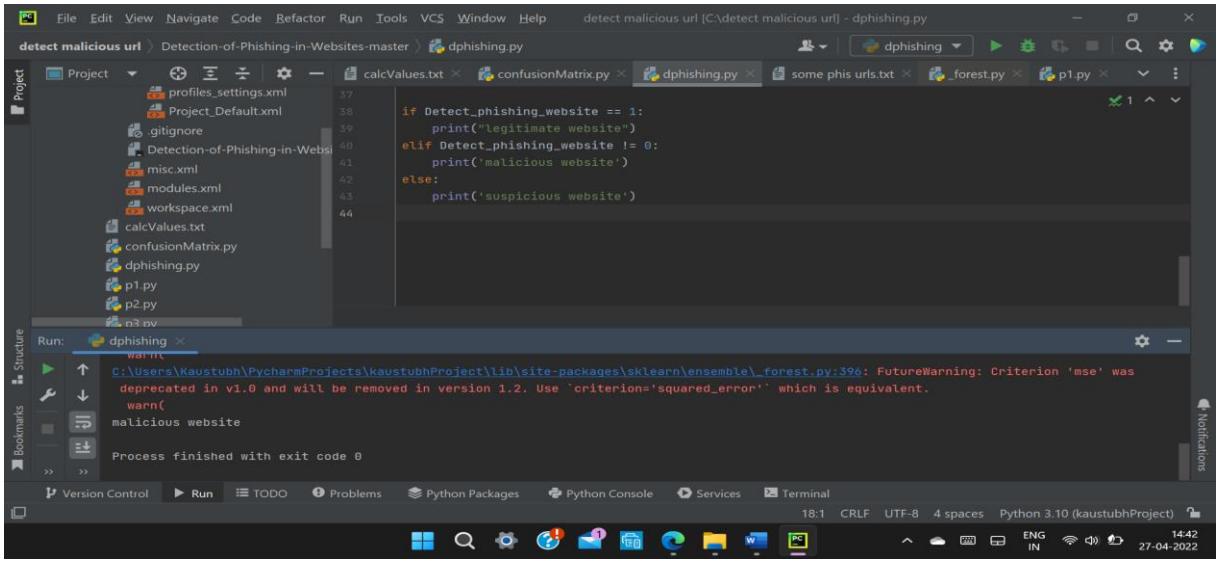


Output Of Malicious URL

Here I runed the program and pasted the copied link or you can use your malicious URL to check. Then I clicked enter button.

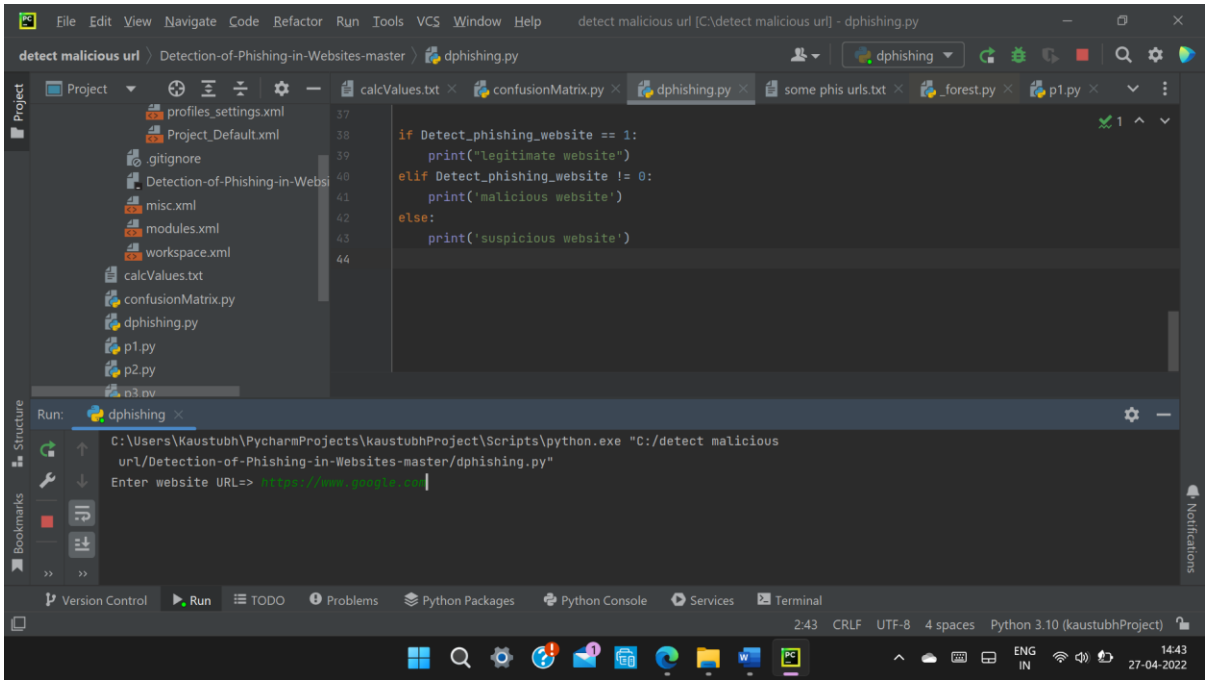


Here is the result when I clicked enter key after pasting / Writing URL Malicious website

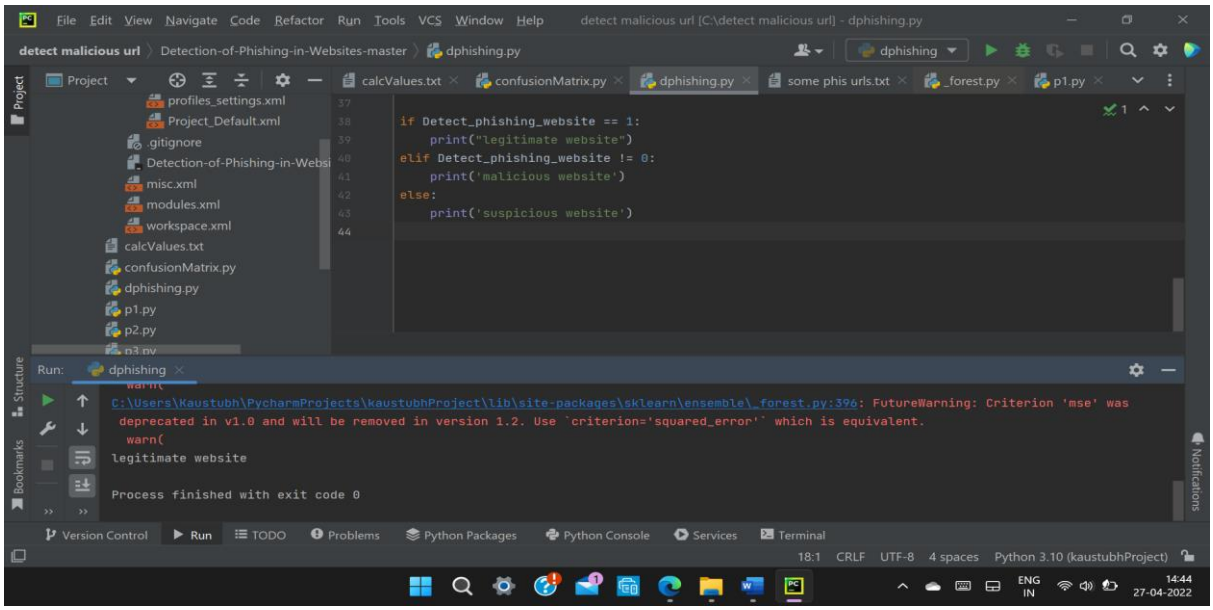


Output Of Legitimate URL

Here I runed the program and pasted the copied link or you can use your legitimate URL to check. Then I clicked enter button



Here is the result when I clicked enter key after pasting / Writing URL Legitimate website



9. CONCLUSION

This focus on the problem of detecting malicious URLs based on the information obtained from URLs string without the need to download page content. We presented and described a variety of features that can be extracted to represent a URL effectively. Then we explained the problem of appropriate representation of features and their possible necessary normalization. The improper representation of data was the reason we did not initially achieve good and desired results. We proposed and implemented a solution that automatically extracts, normalize, and prepare the input data for different machine learning libraries. This implementation is fully modular and can be expanded to support additional features, normalization techniques, or machine learning libraries.

The main objective of this thesis was to assess whether machine learning classification technologies are a suitable solution and can give us a reliable prediction of whether URL is malicious or not. We executed and evaluated our experiments on the models created by data regression library, one representative of batch learning, and one of the online learning approaches. We were interested in differences in results, but also in their efficiency and time-consumption. Surprisingly, both approaches achieved very similar results evaluated on the same input data. When we used all features and a training set of size 352 096 URL samples, both achieved accuracy more than 97.3%.

The presence of the word proved to be a feature that can significantly influence the success of the prediction. Model evaluation without this feature was even not successful or had a bad accuracy. We concluded that data regression probably not an appropriate solution due to its exponentially growing time demand. Even though the analysis has shown that both models can successfully detect new threats after a long period of time, and therefore, the daily re-train of the model is not necessarily needed.

9.1 FUTURE SCOPE:

Due to the use of static datasets for testing and analysis, we could not include the host or rank properties in the list of used features. These properties may change over time, and thus their extraction from the URL after a long time may not reflect the real state at the time of collection. As future work, a tool or way for dynamic data reception can be added, allowing to extract and use more features. Even though the machine-learning technologies we used achieved very good results, comparing multiple models and settings can bring more interesting insights and results.

9.2 Advantage and Disadvantage:

Advantages of Fraud URL detection:

- Users can access the application anywhere anytime.
- The application is available 24x7.
- Application does not lag on a frequent unknown basis.
- Application does not need an internet connection for access.
- It is easy to use.

Disadvantages of Fraud URL Detection:

- Loss of Data. Clicking on a malicious link in an email can hand over the data and system of an organization to a hacker.
- Damaged Reputation.
- Direct Monetary Loss.
- Loss of Productivity.
- Loss of Customers.
- Financial Penalties.
- Intellectual Property Theft.

REFERENCE

1. INC., Webroot. 2019 Webroot Threat Report. Available also from: https: / / www - cdn. webroot . com / 9315 / 5113 / 6179 / 2019 _ Webroot_Threat_Report_US_Online.pdf.

2. CHOI, Hyunsang; ZHU, Bin B.; LEE, Heejo. Detecting Malicious Web Links and Identifying Their Attack Types. In: Proceedings of the 2Nd USENIX Conference on Web Application Development. 2011.

3. SINHA, S.; BAILEY, M.; JAHANIAN, F. Shades of grey: On the effectiveness of reputation-based "blacklists". In: 2008 3rd International Conference on Malicious and Unwanted Software (MALWARE). 2008.

4. GOOGLE. Google Safe Browsing - Blacklist service provided by Google. Available also from: https: / / safebrowsing . google . com/.

5. MALWAREURL. Phishing blacklist operated by OpenDNS. Available also from: https://www.malwareurl.com/.

6. MA, Justin; SAUL, Lawrence; SAVAGE, Stefan; VOELKER, Geoffrey. Beyond blacklists: learning to detect malicious Web sites from suspicious URLs. In: 2009, pp. 1245–1254.

7. SAHOO, Doyen; LIU, Chenghao; HOI, Steven C. H. Malicious URL Detection using Machine Learning: A Survey. 2017.

8. MCGRATH, D.; GUPTA, Minaxi. Behind Phishing: An Examination of Phisher Modi Operandi. In: 2008.

9. KOLARI, Pranam; FININ, Tim; JOSHI, Anupam. SVMs for the Blogosphere: Blog Identification and Splog Detection. In: 2006, pp. 92–99.

10. GARERA, Sujata; PROVOS, Niels; CHEW, Monica; RUBIN, Aviel D. A Framework for Detection and Measurement of Phishing Attacks. In: Proceedings of the 2007 ACM Workshop on Recurring Malcode. 2007.